

Creating Visualisations for the Blog

Below I am setting up colour palettes to use in the visualisations to get consistent mappings throughout the plots.

```
## importing packages
import pandas as pd
import re
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
from IPython.display import HTML

## creating the mapping for countries to be used in both the book and series
plots
palette16 = dict(Chinese = "pink", Dutch = "orchid", English = "thistle", French
= "mediumslateblue", German ="cornflowerblue",
                Italian = "lightsteelblue", Japanese = "lightskyblue", Norwegian
="mediumaquamarine", Polish = "lightgreen", Russian = "lightgoldenrodyellow",
                Swedish = "moccasin", Portuguese = "sandybrown", Spanish = "darksalmon",
Czech = "lightcoral",Yiddish = "silver",
                Gujarati = "darkgrey")

## this below palette I will be using for genres in the book genre plot
palette15 = ( "pink", "orchid", "thistle", "mediumslateblue", "cornflowerblue",
              "lightsteelblue", "lightskyblue", "mediumaquamarine", "lightgreen",
              "lightgoldenrodyellow",
              "moccasin", "sandybrown", "darksalmon", "lightcoral", "silver")
```

This section will focus on visualisations using the individual book data.

```
## loading in the book dataframe and checking the formatting is correct
df_book = pd.read_csv('../data/top_books.csv')
df_book = df_book.drop(df_book.columns[0], axis =1)
df_book
```

	Book	Author	Original_Language	First_Publication_Year	Approximate_Sales	Genre_1	Genre_2	Genre_3	Genre_4	Genre_5	Genre_6	Alternative_Genre	Age_Category
0	A Tale of Two Cities	Charles Dickens	English	1859	2000000	Historical fiction	NaN	NaN	NaN	NaN	NaN	Historical fiction	Adult
1	The Little Prince (Le Petit Prince)	Antoine de Saint-Exupéry	French	1943	2000000	Fantasy	children's fiction	NaN	NaN	NaN	NaN	Fantasy, children's fiction	Children
2	The Alchemist (O Alquimista)	Paulo Coelho	Portuguese	1988	1500000	Fantasy	NaN	NaN	NaN	NaN	NaN	Fantasy	Adult
3	Harry Potter and the Philosopher's Stone	J.K. Rowling	English	1997	1200000	Fantasy	children's fiction	NaN	NaN	NaN	NaN	Fantasy, children's fiction	Children
4	And Then There Were None	Agatha Christie	English	1939	1000000	Mystery	NaN	NaN	NaN	NaN	NaN	Mystery	Adult
...	...	...	...	...	...	...	...	...	...	...	...	...	...
161	Bridget Jones's Diary	Helena Fielding	English	1996	1000000	NaN	NaN	NaN	NaN	NaN	NaN	novel	Adult
162	It Ends with Us	Colleen Hoover	English	2016	1000000	Roman Fiction	NaN	NaN	NaN	NaN	NaN	Roman Fiction	Adult
163	The Lion and the Witch	C.S. Lewis	English	1950	850000	Fantasy	children's	NaN	NaN	NaN	NaN	Fantasy,	Children

	Book	Author	Original_Language	First_Published	Approach_Sales	Genre_1	Genre_2	Genre_3	Genre_4	Genre_5	Genre_6	Al_1_Genre	Age_Cat-
			guage										egory
	and the Wardrobe					fic- tion						chil- dren's fic- tion	
164	The Se- cret Di- ary of Adrian Mole, Aged 13¾	Sue Townsend	English	1982	2000000	Young adult novel	NaN	NaN	NaN	NaN	NaN	Young adult novel	Young Adult
165	Ro- nia, the Rol- ber's Daugh- ter	Astrid Lind- gren	Swedish	1981	1000000	NaN	NaN	NaN	NaN	NaN	NaN	Chil- dren's fic- tion	Chil- dren

```

## Here I am creating a dataframe with the main genres for individual books
# I created a separate dataframe as books can take multiple genres and this was
the best method to reflect this

genre_class = pd.DataFrame({"genre": ["fantasy", "thriller", "novel",
"autobiographical", "biographical", "historical",
"gothic", "self-help", "romance", "mystery", "philosophical",
"mystery", "science fiction",
"non-fiction", "fiction"],
"count": [df_book["All_Genre"].str.contains("fantasy", case =
False).sum(),
df_book["All_Genre"].str.contains("thriller", case = False).sum(),
df_book["All_Genre"].str.contains("novel", case = False).sum(),
df_book["All_Genre"].str.contains("autobiograph", case = False).sum()
+ df_book["All_Genre"].str.contains("memoir", case = False).sum(),
df_book["All_Genre"].str.contains("biograph", case = False).sum(),
df_book["All_Genre"].str.contains("historical", case = False).sum(),
df_book["All_Genre"].str.contains("gothic", case = False).sum(),
df_book["All_Genre"].str.contains("self-help", case = False).sum(),
df_book["All_Genre"].str.contains("romance", case = False).sum()]})

```

```

+ df_book["All_Genre"].str.contains("romantic", case = False).sum(),
    df_book["All_Genre"].str.contains("mystery", case = False).sum(),
    df_book["All_Genre"].str.contains("romance", case = False).sum()
+ df_book["All_Genre"].str.contains("romantic", case = False).sum(),
    df_book["All_Genre"].str.contains("Philosoph", case = False).sum(),
    df_book["All_Genre"].str.contains("science fiction", case =
False).sum(),
    df_book["All_Genre"].str.contains("nonfiction", case = False).sum()
+ df_book["All_Genre"].str.contains("non-fiction", case = False).sum(),
    df_book["All_Genre"].str.contains(" fiction", case = False).sum()
+ (df_book["All_Genre"] == "Fiction").sum()])})

```

```

## Here I am plotting the genres in a pie-chart format

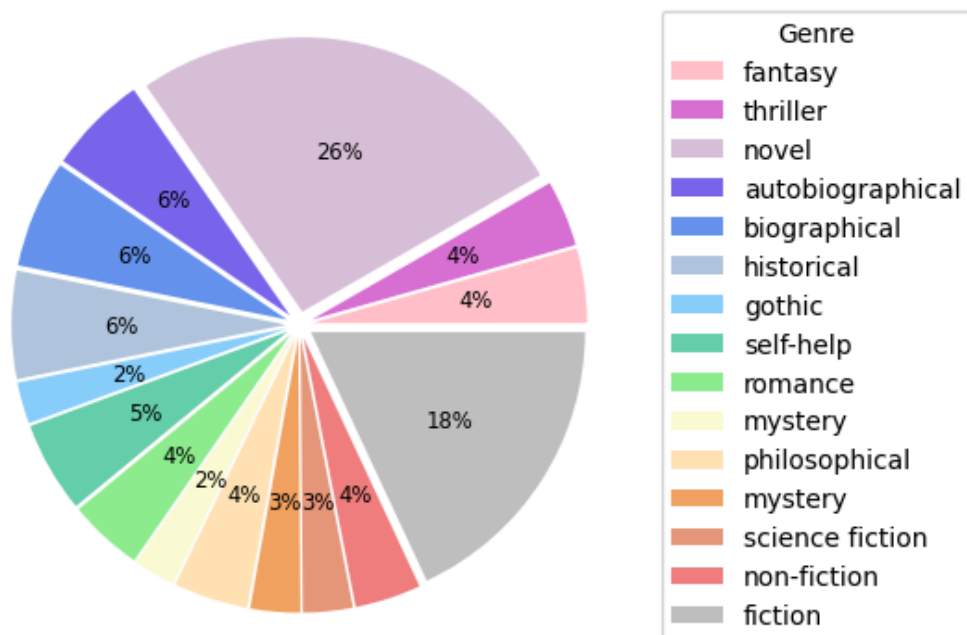
# This part allows for spacing between the wedges for more clarity
explode = (0.05, 0.05, 0.05, 0.05, 0.05, 0.05, 0.05, 0.05, 0.05, 0.05,
0.05, 0.05, 0.05, 0.05)

# Creating pie-chart with percentages
plt.pie(genre_class["count"], colors=palette15, explode = explode,
    autopct='%1.0f%%', textprops={'size': 'small'})
plt.title("Pie Chart of Top Genre Classifications for Individual Books")
plt.legend(loc="center right", fontsize=10, title = "Genre",
bbox_to_anchor=(1.5,0.5),
    labels = genre_class["genre"])

plt.savefig("../results/genre_pie_chart.png", dpi = 200, bbox_inches = "tight")

```

Pie Chart of Top Genre Classifications for Individual Books



```
## This is an interactive plot for the book genres that cannot be used in the
blog as it is incompatible with hackmd
# However, it is a very insightful plot and is good for investigation and other
uses
# It is a slightly more detailed version of the plot above which is compatible
with interactive plots
# but would be overwhelming on a static plot.
fig = px.pie(genre_class, values='count', names='genre',
color="genre",color_discrete_sequence = palette15,
title = "Pie Chart of Top Genre Classifications for Individual Books",
labels = {"genre": "Book Genre", "count": "Number of Books"})

fig.show()
```

Unable to display output for mime type(s): application/vnd.plotly.v1+json

```
## Here I am making two grouped dataframes for languages and age categories of
plots

# This is a sum of sales dataframe
grouped_age_sales = pd.DataFrame(df_book.groupby([ "Original_Language",
"Age_Category"])
```

```
['Approximate_Sales'].sum()).unstack()

# This is a count of books dataframe
grouped_age_count = pd.DataFrame(df_book.groupby([ "Original_Language",
                                                    "Age_Category"]))

["Book"].count()).unstack()
```

```
## Here I am creating a subplot of two bar plots
# The first is the count of books of original languages split by age category
# The second is the sum of sales of original languages split by age category

fig, axs = plt.subplots(ncols=2, figsize=(20, 7))

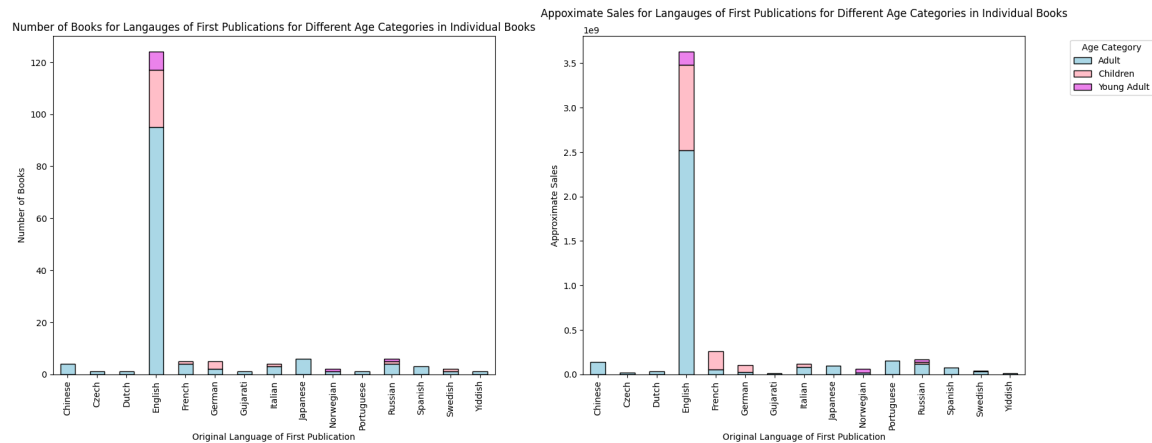
grouped_age_count.plot(kind = 'bar', stacked = True, color = ["lightblue",
"pink", "violet"],
                        legend=False, edgecolor = "black", ax = axs[0]).set(
    xlabel = "Original Language of First Publication",
    ylabel = "Number of Books",
    title = "Number of Books for Languages of First Publications for
Different Age Categories in Individual Books"
)

grouped_age_sales.plot(kind = 'bar', stacked = True, color = ["lightblue",
"pink", "violet"],
                        legend=False, edgecolor = "black", ax = axs[1]).set(
    xlabel = "Original Language of First Publication",
    ylabel = "Approximate Sales"
)

plt.legend(["Adult", "Children", "Young Adult"], loc='upper right',
bbox_to_anchor=(1.3, 1), title = "Age Category")

plt.title("Appoximate Sales for Languages of First Publications for Different
Age Categories in Individual Books", y =1.04)

plt.savefig("../results/book_language_age.png", bbox_inches = "tight")
```



```
## Here I am making a subplot of two scatter graphs
# They are of years of first published against approximate sales
# The colours map to the original language
# The second plot is a subsection of the first plot

fig, axs = plt.subplots(ncols=2, figsize=(15, 7))
sns.scatterplot(data=df_book, y="Approximate_Sales", x="First_Published",
                hue="Original_Language", palette = palettel6, ax = axs[0],
                legend=False).set(
    xlabel = "Year of First Publication", ylabel = "Approximate
Sales"
)

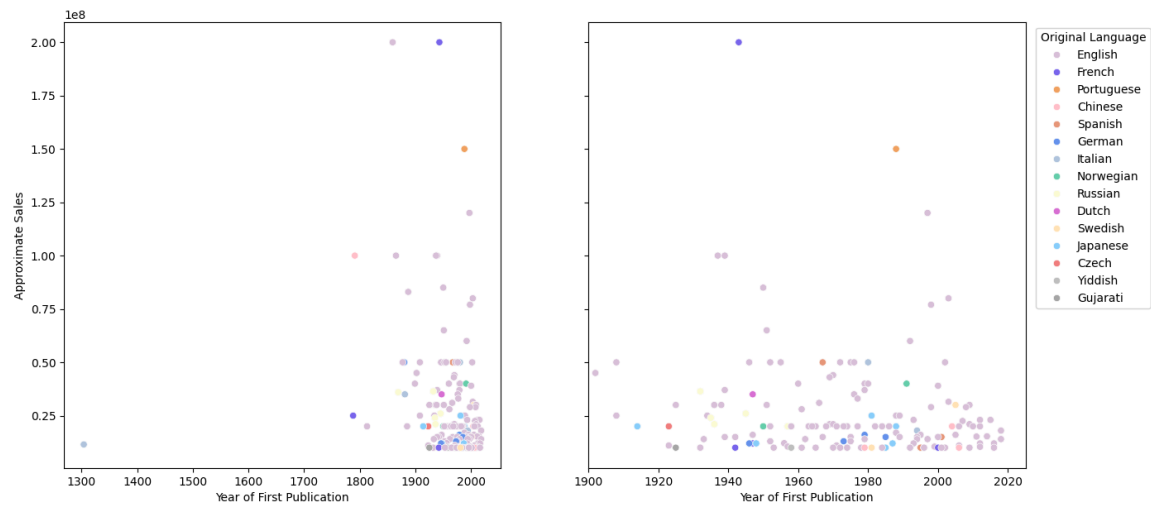
sns.scatterplot(data=df_book, y="Approximate_Sales", x="First_Published",
                hue="Original_Language", palette = palettel6, ax = axs[1]).set(
    xlabel = "Year of First Publication",
    yticklabels=[],
    ylabel = None,
    xlim = [1900, 2025]
)

plt.legend(loc='upper right', bbox_to_anchor=(1.3, 1), title = "Original
Language")

fig.suptitle("Approximate Sales across Years of Publications for Individual
Books")

plt.savefig("../results/book_language_scatter.png", bbox_inches = "tight")
```

Approximate Sales across Years of Publications for Individual Books



```
## This is an interactive plot of above that cannot be used in the blog as it
is incompatible with hackmd
# However, it is a very insightful plot and is good for investigation and other
uses
# It is a slightly more detailed version of the plot above which is compatible
with interactive plots
# but would be overwhelming on a static plot.
```

```
fig2 = px.scatter(df_book, x="First_Published", y="Approximate_Sales",
                  color="Original_Language", color_discrete_map=palettel6,
                  title="Approximate Sales across Years of Publications for Individual
Books",
                  hover_data=["Book", "Author"],
                  labels={ "First_Published": "Year of First Publication",
                          "Original_Language": "Original Language of Publication",
                          "Approximate_Sales": "Approximate Sales"
                  })

fig2.show()
```

Unable to display output for mime type(s): application/vnd.plotly.v1+json

```
## Here I am making a cumulative sum dataframe of the different age categories
across years
```

```
children = df_book[df_book["Age_Category"] == "Children"][["First_Published",
"Approximate_Sales"]
                                                         ].groupby("First_Published").sum().cumsum().reset_index()
children["Age_Category"] = "Children"
```



```

young_adult = df_book[df_book["Age_Category"] == "Young Adult"]
[["First_Published", "Approximate_Sales"]
].groupby("First_Published").sum().cumsum().reset_index()
young_adult["Age_Category"] = "Young Adult"

adult = df_book[df_book["Age_Category"] == "Adult"][["First_Published",
"Approximate_Sales"]
].groupby("First_Published").sum().cumsum().reset_index()
adult["Age_Category"] = "Adult"

# Here I am adding the maximum amounts as the last datapoint to make sure the
sums finish at the same point
final_entries = pd.DataFrame({"First_Published": [2020, 2020, 2020],
                              "Approximate_Sales": [1312000000, 2090000000,
3383900000],
                              "Age_Category": ["Children", "Young Adult",
"Adult"]})

age_range_sales = pd.concat([children, young_adult, adult, final_entries])

```

```

## Here I am plotting the cummualtive sums of sales of the three age categories
# This is a line graph as it is best suited to a time-series analysis

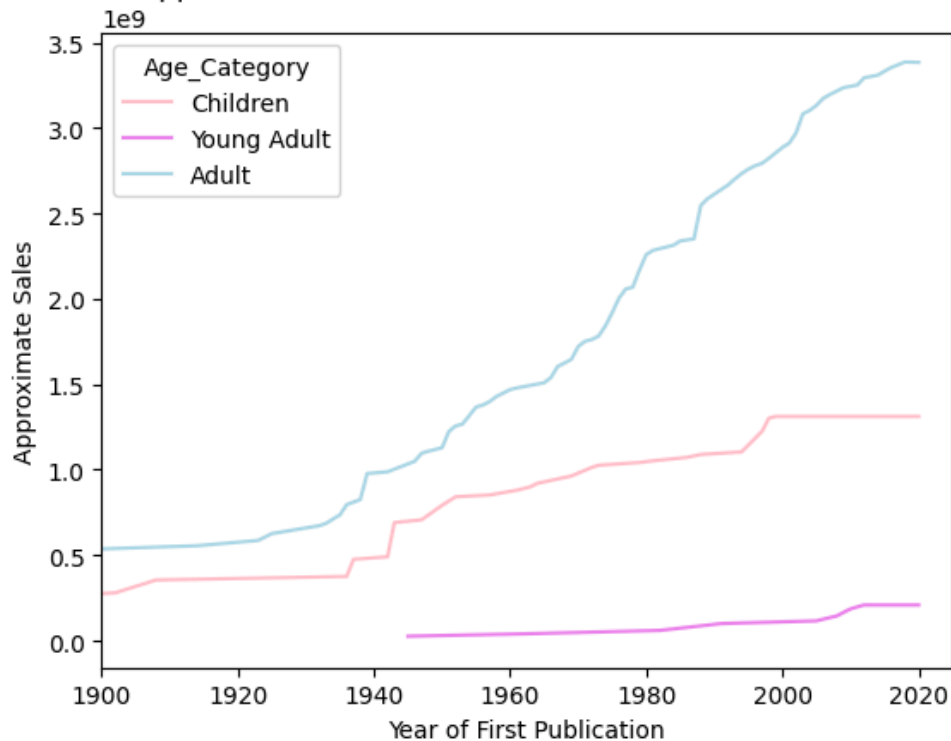
sns.lineplot(age_range_sales, x="First_Published", y="Approximate_Sales",
             hue="Age_Category", palette = ["pink", "violet", "lightblue"]).set(
    xlabel = "Year of First Publication",
    ylabel = "Approximate Sales",
    xlim = [1900, 2025]
)

plt.title( "Cumulative Appoximate Sales across Years of Publications for
Individual Books", y =1.04)

plt.savefig("../results/cumulative_sales_plot.png", bbox_inches = "tight")

```

Cumulative Appoximate Sales across Years of Publications for Individual Books



Within this section, I will be creating plots for the series dataframe.

```
## Here I am loading in the book series dataframe and checking the format
df_series = pd.read_csv('./data/top_series.csv')
df_series = df_series.drop(df_series.columns[0], axis =1)
df_series
```

Book_Series	Author	Original_Language	Years_of_Publication	Genre	Genre_Prox	First_Publication	Last_Publication	No_of_Books	Extras	Extrapolated	Al-Num	Num_of_Publications
0	Harry Potter	English	7 + 3	Children's fiction	Children's fiction	1997	2007	7.0	7.0	True	NaN	10
1	Goosebumps	English	62 + 1	Children's fiction	Children's fiction	1992	2025	62.0	NaN	True	NaN	33

Book_Series	Author	Original_Language	Years_of_Publication	Initial_Sales	Genre	Genre2	First_Year_Published	Last_Year_Published	No_of_Books	Extras	Extrabooks	Years_Published	Num-lication
2	Perry Erle Ma-son	Eng-lish	82	1933-3000000000	off series	NaN	1933	1973	82.0	4.0	True	NaN	40
3	Di-ary of a Wimpy Kid	Eng-lish	19	2007-2900000000	off series	NaN	2007	2025	19.0	5.0	True	NaN	18
4	Berenstain Bears	Eng-lish	428	1962-2600000000	off series	NaN	1962	2025	428.0	NaN	False	NaN	63
123	Hor-ri-ble His-tories	Eng-lish	24	1993-2000000000	off series	NaN	1993	2025	24.0	NaN	False	NaN	32
124	Rain-bow Magic	Eng-lish	80	2003-2000000000	off series	NaN	2003	2025	80.0	NaN	False	NaN	22
125	Mor-gan Kane	Nor-we-gian	90	1966-2000000000	off series	NaN	1966	2025	90.0	NaN	False	NaN	59
126	The South-ern Vampires	Eng-lish	13	2001-2000000000	off series	NaN	2001	2013	13.0	NaN	False	NaN	12

Book_Series	Author	Original_Language	Year_of_Publication	Approximate_Sales	Genre	Genre2	First_Installment_Published	No_of_Installments	Ex-tras	Ex-betras	Al-Num
127	鬼平犯科帳 (Oni-hei Han-kachō)	Shōtarō Ikenami	Japanese	24	1968-1990	24400000	1968	1990	24.0	NaN	False

```

## Here I am creating a subplot with one histogram and two scatter graphs
# They use the same colour schemes that are used within the individual book
visualisations

fig, axs = plt.subplots(ncols=3, figsize=(18, 5))

sns.histplot(df_series, x = "Number_of_Years_Publication", color ="pink" , ax =
axs[0]).set(
    xlabel = "Number of Years from First to Last Publication",
    ylabel = "Count of Series",
    title = "Count of Number of Years from First to Last Publication")

sns.scatterplot(data=df_series, y="Approximate_Sales",
x="First_Installment_Published",
hue="Original_Language", palette = palette16, ax=axs[1],
legend=False).set(
    xlabel = "Year of First Installment Publication",
    ylabel = "Approximate Sales",
)

sns.scatterplot(data=df_series, y="Approximate_Sales",
x="Last_Installment_Published",
hue="Original_Language", palette = palette16, ax = axs[2]).set(
    yticklabels=[],
    ylabel = None,
    xlabel = "Year of Last Installment Publication"
)

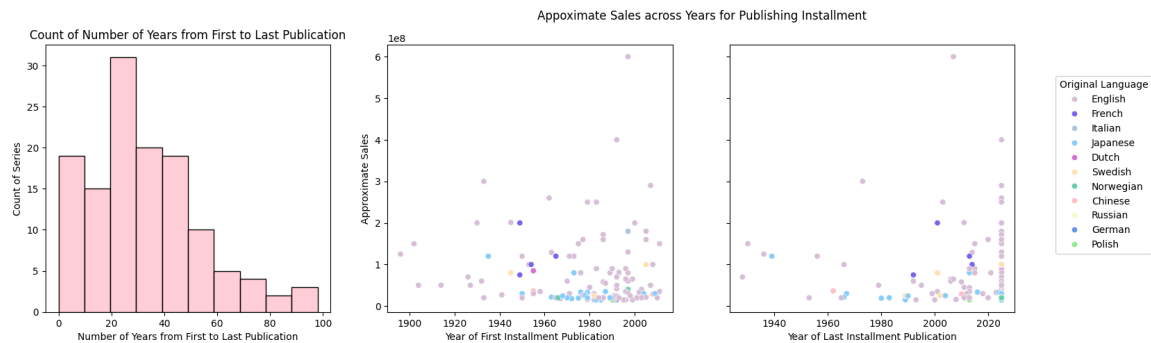
plt.legend(loc='upper right', bbox_to_anchor=(1.5, 0.9), title = "Original
Language")

plt.suptitle( "Appoximate Sales across Years for Publishing Installment", x =

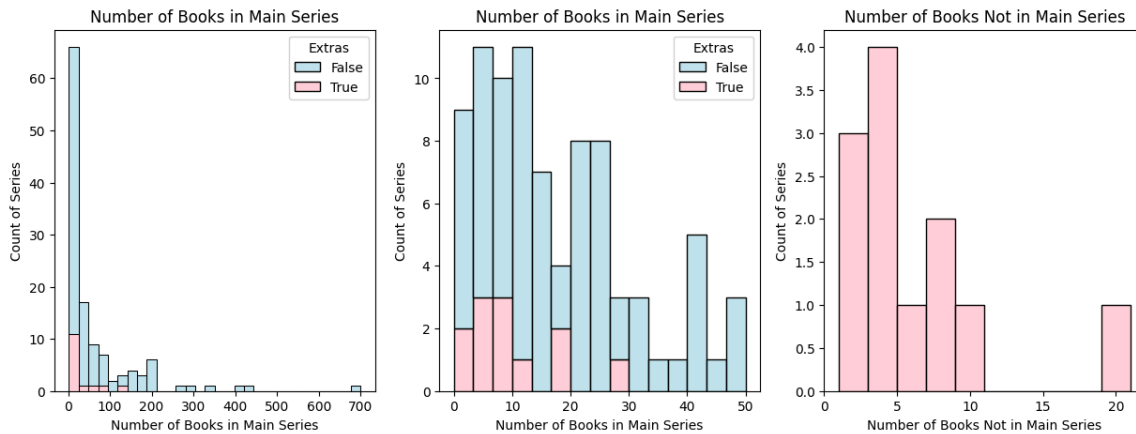
```

0.65)

```
plt.savefig("../results/first_to_last_installment_plot.png",    bbox_inches =  
"tight")
```



```
## Here I am making a subplot of three histograms  
# They are all related to number of book fields  
# The last two use the Boolean field for extra books as a colour category  
  
fig, axs = plt.subplots(ncols=3, figsize=(15, 5))  
  
sns.histplot(df_series, x = "No_of_Main_Books", stat = "count",  
             bins = 30, hue='Extras', multiple='stack', palette=['lightblue',  
             'pink'],  
             ax=axs[0]).set(  
             xlabel = "Number of Books in Main Series",  
             ylabel = "Count of Series",  
             title = "Number of Books in Main Series")  
  
sns.histplot(df_series, x = "No_of_Main_Books", stat = "count",  
             bins = 15, binrange= [0,50], hue='Extras', multiple='stack',  
             palette=['lightblue', 'pink'],  
             ax=axs[1]).set(  
             xlabel = "Number of Books in Main Series",  
             ylabel = "Count of Series",  
             title = "Number of Books in Main Series")  
  
sns.histplot(df_series, x = "No_of_Extras", color = "pink" , bins = 10, ax =  
             axs[2]).set(  
             xlabel = "Number of Books Not in Main Series",  
             ylabel = "Count of Series",  
             title = "Number of Books Not in Main Series")  
  
plt.savefig("../results/number_books_series.png", bbox_inches = "tight")
```



```
## Here I am creating a summary table on number of books in the series

table_series = df_series[["No_of_Main_Books",
                           "No_of_Extras"]].describe().round(1)
table_series.columns = ["Number of Main Books", "Number of Extra Books"]
table_series = table_series.transpose()
table_series.columns = ["Count of Non Null Values", "Mean", "Standard Deviation",
                        "Minimum", "Lower Quantile (25%)",
                        "Median (50%)", "Upper Quantile (75%)", "Maximum"]

table_series.to_csv('../results/table_series.csv')
```

This last section combines a plot for the series and individual books.

```
## Here I am making a subplots of 4 bar graphs
# Two are from the individual book data
# The other two are from the book series data
# They are made in a similar format to eachother
# I create two grouped tables to get a sum plot and the main dataframes to get
plots of the mean sales

fig, axs = plt.subplots(nrows=2, ncols=2, figsize=(19, 7))

grouped_sales_language = df_book.groupby("Original_Language").sum()
g1 = sns.barplot(grouped_sales_language, x = "Original_Language", y =
                  "Approximate_Sales", hue = "Original_Language",
                  palette = palettel6, legend = False, errorbar=None, ax = axs[0, 0],
                  order =
df_book.sort_values("Original_Language").Original_Language).set(
    xticklabels=[],
    xlabel = None,
    ylabel = "Approximate Sales",
    title = "Total Approximate Sales for Top Individual Books per
```

```

Original Language")

g2 = sns.barplot(df_book, x = "Original_Language", y = "Approximate_Sales", hue
= "Original_Language",
                  palette= palettel6, legend = False, order =
df_book.sort_values("Original_Language").Original_Language,
                  ax = axs[1, 0] )

g2.tick_params(axis = 'x', rotation=90)

g2.set( xlabel = "Original Language of Publication",
        ylabel = "Approximate Sales",
        title = "Average Appoximate Sales for Top Individual Books per
Original Language"
        )

grouped_sales_language = df_series.groupby("Original_Language").sum()
g3 = sns.barplot(grouped_sales_language, x = "Original_Language", y =
"Approximate_Sales", hue = "Original_Language",
                  palette = palettel6, legend = False, errorbar=None, ax = axs[0, 1],
                  order =
df_series.sort_values("Original_Language").Original_Language).set(
    xticklabels=[],
    xlabel = None,
    ylabel = "Approximate Sales",
    title = "Total Appoximate Sales for Top Series per Original
Language")

g4 = sns.barplot(df_series, x = "Original_Language", y = "Approximate_Sales",
hue = "Original_Language",
                  palette= palettel6, legend = False, order =
df_series.sort_values("Original_Language").Original_Language,
                  ax = axs[1, 1] )

g4.tick_params(axis = 'x', rotation=90)

g4.set(xlabel = "Original Language of Publication",
        ylabel = "Approximate Sales",
        title = "Average Appoximate Sales for Top Series per Original
Language")
plt.savefig("../results/sales_per_language.png", bbox_inches = "tight")

```

